



## Java Interview Questions (Easy to Hard)

### 1. Basic Operators & Datatypes (15 Questions) - *Fundamentals of Java*

1. What are data types in Java? What are the two main categories?
2. Explain Primitive data types in Java with their sizes and default values.
3. What is the range of `byte`, `short`, `int`, and `long`?
4. What is a `char` data type?
5. What is a `boolean` data type?
6. What is the difference between `float` and `double`?
7. Explain `type casting` (widening and narrowing conversion).
8. What are operators in Java? List different types of operators.
9. Explain Arithmetic operators with examples.
10. Explain Relational operators with examples.
11. Explain Logical operators with examples.
12. Explain Assignment operators with examples.
13. Explain Unary operators (`++`, `--`, `!`).
14. What is the Ternary operator (`? :` )?
15. Explain Bitwise operators.

### 2. Arrays (15 Questions) - *Basic Data Structures*

16. What is an Array in Java?
17. How do you declare and initialize an array in Java?
18. What is the default value of array elements?
19. Can arrays store elements of different data types?
20. How do you access elements in an array?
21. What is `ArrayIndexOutOfBoundsException`?
22. How do you find the length of an array?
23. Differentiate between Array and `ArrayList`.
24. What are multidimensional arrays? How are they declared and initialized?
25. What is a "jagged array" in Java?
26. How do you copy an array in Java?
27. How do you iterate over an array?
28. Can you resize an array after it's created?
29. Explain `Arrays.sort()` and `Arrays.binarySearch()`.
30. How do you convert an array to a `List`?



### 3. Lang Package (20 Questions) - *Core Language Features*

31. What is `java.lang` package? Why is it implicitly imported?
32. What is `Object` class? What are its important methods?
33. Explain `equals()` and `hashCode()` methods of `Object` class.
34. Explain `toString()` method. What is its default implementation?
35. What is `getClass()` method?
36. What is `String` in Java? Is it mutable or immutable? Why?
37. What is String Pool?
38. Differentiate between `String`, `StringBuilder`, and `StringBuffer`. When to use which?
39. What are Wrapper Classes? Why are they used?
40. Explain Autoboxing and Unboxing.
41. Explain the `System` class. What are its uses?
42. What is `Math` class?
43. Explain `Number` class.
44. What is `Void` class?
45. What is `Runnable` interface (from `java.lang`)?
46. What is `Comparable` interface (from `java.lang`)?
47. Explain `clone()` method and the `Cloneable` interface. What is shallow vs. deep copy?
48. What is `Runtime` class?
49. What is `Thread` class (from `java.lang`)?
50. What is `Process` class?

### 4. OOPS Concepts (20 Questions) - *Pillars of Java*

51. What is OOPs? Explain its core principles.
52. What is a Class in Java? How is it different from an Object?
53. What is a Constructor? What are its types?
54. What is the `this` keyword in Java?
55. What is the `super` keyword in Java?
56. Explain the concept of Encapsulation with an example.
57. Explain Inheritance in Java. What are its types?
58. Differentiate between IS-A relationship and HAS-A relationship.
59. What is Polymorphism? Explain Compile-time (Method Overloading) and Run-time (Method Overriding) Polymorphism.
60. Can you overload the main method? Why or why not?



61. Can you override a static method? Why or why not?
62. Explain the concept of **final** keyword in Java (with variables, methods, and classes).
63. What are Access Modifiers in Java? Explain **public**, **private**, **protected**, and default.
64. What is a **static** keyword in Java? Explain its usage with variables, methods, and blocks.
65. What is Abstraction? How is it achieved in Java?
66. Differentiate between Abstract Class and Interface. When would you use one over the other?
67. What is **instanceof** operator? Provide an example.
68. What is Association, Aggregation, and Composition? Give examples.
69. Explain the concept of "coupling" and "cohesion" in OOP.
70. What is a "POJO" (Plain Old Java Object)?

## 5. Exception Handling (20 Questions) - *Robustness in Java*

71. What is an Exception in Java?
72. What is Exception Handling? Why is it important?
73. Explain the Java Exception Hierarchy.
74. Differentiate between **Error** and **Exception**.
75. Differentiate between **Checked Exception** and **Unchecked Exception** (Runtime Exception).
76. Give examples of checked and unchecked exceptions.
77. Explain **try**, **catch**, **finally** blocks.
78. Can you have multiple **catch** blocks? In what order should they be placed?
79. What is the purpose of the **finally** block? Does it always execute?
80. What is **NullPointerException**? How to avoid it?
81. What is **ArrayIndexOutOfBoundsException**?
82. What is **ClassCastException**?
83. What is the **throws** keyword? When do you use it?
84. What is the **throw** keyword? Differentiate **throw** and **throws**.
85. What is **printStackTrace()**?
86. What is **getMessage()**?
87. How do you create custom exceptions in Java?
88. What is Exception Propagation?
89. What is Exception Chaining?
90. What is **try-with-resources**? Explain its benefits.



## 6. Collection API (30 Questions) - *Standard Data Structures & Algorithms*

91. What is the Java Collections Framework? What are its benefits?
92. Explain the hierarchy of the Java Collection Framework.
93. What are the core interfaces in the Collections Framework?
94. Differentiate between `Collection` and `Collections` in Java.
95. Explain `List` interface and its common implementations (`ArrayList`, `LinkedList`, `Vector`).
96. Differentiate between `ArrayList` and `LinkedList`. When to use which?
97. Differentiate between `ArrayList` and `Vector`.
98. What is a `Set` interface? Name its common implementations (`HashSet`, `LinkedHashSet`, `TreeSet`).
99. Can a `Set` contain duplicate elements? Can it contain `null`?
100. Differentiate between `HashSet` and `LinkedHashSet`.
101. Differentiate between `HashSet` and `TreeSet`.
102. What is a `Map` interface? Name its common implementations (`HashMap`, `LinkedHashMap`, `TreeMap`).
103. Can a `HashMap` contain `null` keys/values? How many?
104. Differentiate between `HashMap` and `LinkedHashMap`.
105. Differentiate between `HashMap` and `TreeMap`.
106. Differentiate between `HashMap` and `Hashtable`.
107. Explain `Iterator` in Java. What are its methods?
108. Differentiate between `Iterator` and `ListIterator`.
109. What is `Enumeration`? How is it different from `Iterator`?
110. What is `Queue` interface? Name its common implementations.
111. What is a `Deque`?
112. Explain `PriorityQueue`.
113. How do you sort a `List` of custom objects?
114. Explain `Comparable` and `Comparator` interfaces. Differentiate them.
115. What is `Collections.sort()` and `Arrays.sort()`?
116. What is the `hashCode()` method, and how does it relate to the `equals()` method in collections?
117. What happens if you override `equals()` but not `hashCode()`?
118. What are "fail-fast" and "fail-safe" iterators? Give examples.
119. What is `Stack`? Is it recommended to use it?
120. What is `Dictionary`? Is it recommended to use it?



## **7. Util Package (20 Questions) - *Utility Classes & Modern Features***

121. What is `java.util` package? What kind of classes does it contain?
122. Explain `Scanner` class. How do you read input from the console?
123. Explain `Random` class.
124. Explain `Date` and `Calendar` classes. What are their limitations?
125. What is `UUID` class?
126. What is `Timer` and `TimerTask`?
127. What are the key interfaces in `java.util.function` package (introduced in Java 8)?
128. Explain `Predicate`, `Consumer`, `Supplier`, `Function` functional interfaces.
129. What is `Optional` class (from `java.util`)?
130. Explain `StringTokenizer` class.
131. What is `Formatter` class?
132. What is `Properties` class?
133. What is `Locale` class?
134. What is `Collections` utility class (from `java.util.Collections`)?
135. What is `Arrays` utility class (from `java.util.Arrays`)?
136. What is `Objects` utility class?
137. What is `Currency` class?
138. What is `ResourceBundle`?
139. What are `ConcurrentHashMap` and `CopyOnWriteArrayList`? When would you use them? (Also relates to `Threads`)
140. Explain `WeakHashMap`. (Relates to `Collections`, but more advanced usage)

## **8. Java 8 Features (20 Questions) - *Modern Java Paradigms***

141. What are the major new features introduced in Java 8?
142. Explain Lambda Expressions in Java 8 with an example.
143. What is a Functional Interface? Give examples of built-in functional interfaces.
144. Explain Method References in Java 8. What are the different types?
145. What is Nashorn JavaScript Engine?
146. Explain the new Date and Time API (`java.time` package). What problems does it solve?
147. What is `LocalDate`, `LocalTime`, `LocalDateTime`?
148. What is `Instant`?



- 149. What is **Duration** and **Period**?
- 150. What is **StringJoiner**?
- 151. What is the **Stream API**? What are its benefits over traditional collections processing?
- 152. Differentiate between **Collections** and **Streams**.
- 153. What are the characteristics of **Java Streams**?
- 154. Is a **Stream** reusable? Why or why not?
- 155. Explain **Default Methods** in **Interfaces**. Why were they introduced?
- 156. Explain **Static Methods** in **Interfaces**.
- 157. What is the **Optional** class in **Java 8**? Why is it useful? (Duplicated with **Util** but important here)
- 158. Explain **ConcurrentHashMap** improvements in **Java 8**.
- 159. What are **ConcurrentHashMap** and **CopyOnWriteArrayList**? When would you use them? (Reiteration from **Collections** for a reason)
- 160. Explain **WeakHashMap**. (Reiteration from **Collections** for a reason)

## **9. Stream API (20 Questions) - *Functional Programming with Collections***

- 161. How do you create a **Stream**?
- 162. Explain **Intermediate Operations** and **Terminal Operations** in **Streams**. Give examples.
- 163. Explain the **filter()** method in **Stream API**.
- 164. Explain the **map()** method in **Stream API**.
- 165. Differentiate between **map()** and **flatMap()**. When would you use **flatMap()**?
- 166. Explain the **forEach()** method.
- 167. Explain **limit()** and **skip()** operations.
- 168. Explain **sorted()** operation.
- 169. What is **distinct()** operation?
- 170. Explain **anyMatch()**, **allMatch()**, **noneMatch()**.
- 171. Explain the **reduce()** method.
- 172. Explain the **collect()** method and its common **Collectors**.
- 173. How do you convert a **Stream** to an **Array**?
- 174. What are primitive streams (**IntStream**, **LongStream**, **DoubleStream**)?
- 175. How do you process a **Stream** in parallel? What are the considerations?
- 176. What is the **Optional** in **Java 8**? Why is it useful? (Reiteration from **Java 8 Features**, emphasis on stream usage)
- 177. How to handle **NullPointerException** using **Optional**?
- 178. How does **Stream.ofNullable()** work?
- 179. What is **peek()** method in **Streams**?
- 180. How to debug **Stream** pipelines?



## 10. Threads (20 Questions) - *Concurrency & Performance*

181. What is a Thread in Java?
182. What is Multithreading? What are its advantages?
183. Differentiate between a Process and a Thread.
184. What are the two ways to create a Thread in Java? Explain with examples.
185. What is the `start()` method and `run()` method in `Thread` class?
186. Can you call the `run()` method directly? What happens if you do?
187. Explain the life cycle of a Thread in Java.
188. What is `Daemon Thread`? How is it different from a user thread?
189. How do you handle exceptions in threads?
190. What is Thread Synchronization? Why is it needed?
191. Explain the `synchronized` keyword (method and block).
192. Differentiate between `wait()`, `notify()`, and `notifyAll()` methods.
193. What is `Race Condition`? How can it be prevented?
194. What is `Deadlock`? How can it be avoided?
195. What is `volatile` keyword in Java?
196. What is `ExecutorService`? What are its benefits?
197. Differentiate between `Runnable` and `Callable` interfaces.
198. What is `Future` in Java concurrency?
199. What is a `ThreadPool`? What are its advantages?
200. Explain `CountDownLatch` and `CyclicBarrier`. (Advanced Concurrency Utilities)
201. What is `Semaphore`?
202. What is `ReentrantLock`? How is it different from `synchronized`?
203. What is `ThreadLocal`? When would you use it?
204. What is `ThreadGroup`?
205. What is context switching?